

Assignment-8

Ht No: 2303A51694

Name: Niharika

Batch-24

Task Description #1 (Username Validator – Apply AI in Authentication Context)

- Task: Use AI to generate at least 3 assert test cases for a function

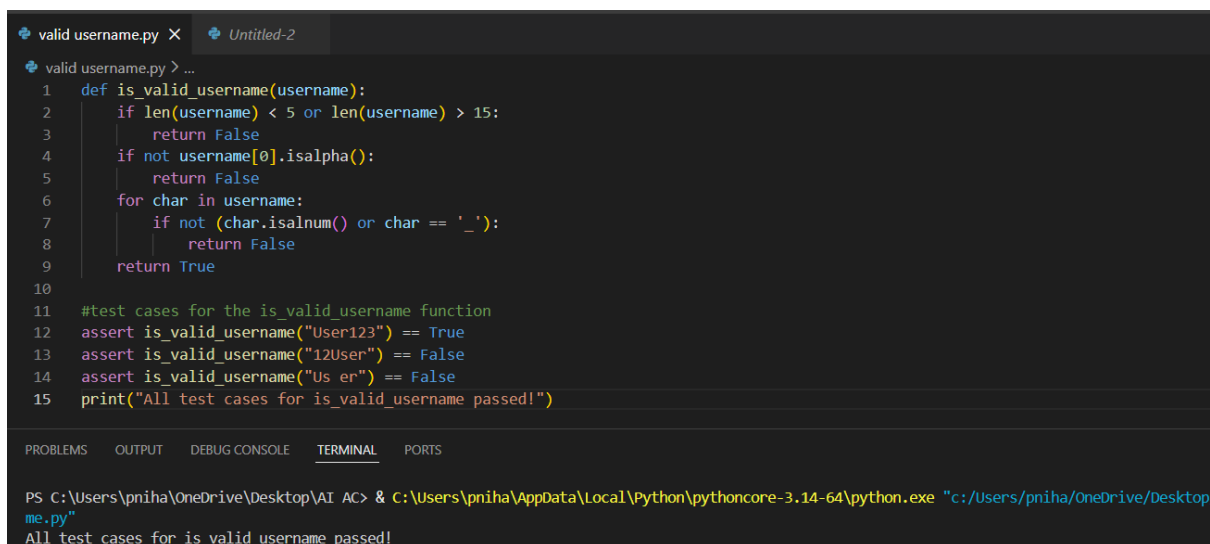
is_valid_username(username) and then implement the function using Test-Driven Development principles.

- Requirements:
 - o Username length must be between 5 and 15 characters.
 - o Must contain only alphabets and digits.
 - o Must not start with a digit.
 - o No spaces allowed. Example Assert Test Cases:

```
assert is_valid_username("User123") == True
assert is_valid_username("12User") == False
assert is_valid_username("Us er") == False
```

Expected Output #1:

- Username validation logic successfully passing all AI-generated test cases.



```
valid username.py X  Untitled-2
valid username.py > ...
1 def is_valid_username(username):
2     if len(username) < 5 or len(username) > 15:
3         return False
4     if not username[0].isalpha():
5         return False
6     for char in username:
7         if not (char.isalnum() or char == '_'):
8             return False
9     return True
10
11 #test cases for the is_valid_username function
12 assert is_valid_username("User123") == True
13 assert is_valid_username("12User") == False
14 assert is_valid_username("Us er") == False
15 print("All test cases for is_valid_username passed!")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\pniha\OneDrive\Desktop\AI AC> & C:\Users\pniha\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/pniha/OneDrive/Desktop/me.py"
All test cases for is_valid_username passed!
```

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

- Task: Use AI to generate at least 3 assert test cases for a function classify_value(x) and implement it using conditional logic and loops.
- Requirements:

- o If input is an integer, classify as "Even" or "Odd".
- o If input is 0, return "Zero".
- o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

```
assert classify_value(8) == "Even"
assert classify_value(7) == "Odd"
assert classify_value("abc") == "Invalid Input"
```

Expected Output #2:

- Function correctly classifying values and passing all test cases.

The screenshot shows a code editor with a file named 'classify_value.py'. The code defines a function 'classify_value(x)' that returns 'Negative', 'Zero', 'Even', or 'Odd' based on the input. Below the function, there are three assert test cases: 'assert classify_value(8) == "Even"', 'assert classify_value(7) == "Odd"', and 'assert classify_value("abc") == "Invalid Input"'. The terminal output shows the execution of the script, which results in a 'TypeError: < not supported between instances of 'str' and 'int'' error, indicating that the function is not handling non-numeric input correctly.

```
def classify_value(x):
    if x < 0:
        return "Negative"
    elif x == 0:
        return "Zero"
    elif x%2 == 0:
        return "Even"
    else:
        return "Odd"

# Test cases for the classify_value function
assert classify_value(8) == "Even"
assert classify_value(7) == "Odd"
assert classify_value("abc") == "Invalid Input"
```

```
PS C:\Users\pniha\OneDrive\Desktop\AI AC> & C:\Users\pniha\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/pniha/OneDrive/De
Traceback (most recent call last):
  File "c:/Users/pniha/OneDrive/Desktop/AI AC/classify_value.py", line 14, in <module>
    assert classify_value("abc") == "Invalid Input"
    ~~~~~^~~~~~
  File "c:/Users/pniha/OneDrive/Desktop/AI AC/classify_value.py", line 2, in classify_value
    if x < 0:
    ~~~~^
TypeError: '<' not supported between instances of 'str' and 'int'
```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

- Task: Use AI to generate at least 3 assert test cases for a function is_palindrome(text) and implement the function.
- Requirements:

o Ignore case, spaces, and punctuation. o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

```
assert is_palindrome("Madam") == True
assert is_palindrome("A man a plan a canal Panama") == True
```

```
assert is_palindrome("Python") == False
```

```
palindrome.py > ...
1 def is_palindrome(text):
2     cleaned_text = ''.join(char.lower() for char in text if char.isalnum())
3     return cleaned_text == cleaned_text[::-1]
4
5 # Test cases for the is_palindrome function
6 assert is_palindrome("Madam") == True
7 assert is_palindrome("A man a plan a canal Panama") == True
8 assert is_palindrome("Python") == False
9 print("All test cases for is_palindrome passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\pniha\OneDrive\Desktop\AI AC> & C:\Users\pniha\AppData\Local\Python\pythoncore-3.14-64\py
All test cases for is_palindrome passed!
PS C:\Users\pniha\OneDrive\Desktop\AI AC> |
```

Task Description #4 (Email ID Validation – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for a function `validate_email(email)` and implement the function.
- Requirements:

o Must contain @ and . o Must not start or end with special characters. o Should handle invalid formats gracefully.

Example Assert Test Cases:

```
assert validate_email("user@example.com") == True
assert validate_email("userexample.com") == False
assert validate_email("@gmail.com") == False
```

Expected Output #5:

- Email validation function passing all AI-generated test cases and handling edge cases correctly.

```
email.py > ...
1 def validate_email(email):
2     if '@' not in email or '.' not in email:
3         return False
4     at_index = email.index('@')
5     dot_index = email.rindex('.')
6     if at_index < 1 or dot_index < at_index + 2 or dot_index >= len(email) - 1:
7         return False
8     return True
9
10 # Test cases for the validate_email function
11 assert validate_email("user@example.com") == True
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\pniha\OneDrive\Desktop\AI AC> & C:\Users\pniha\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Us
All test cases for validate_email passed!
```

Task 5 (Perfect Number Checker – Test Case Design)

- Function: Check if a number is a perfect number (sum of divisors = number).

- Test Cases to Design:
 - o Normal case: 6 → True, 10 → False.
 - o Edge case: 1.
 - o Negative number case.
 - o Larger case: 28.
- Requirement: Validate correctness with assertions.

```

perfect num.py > ...
1  #generate a python code to display whether the given number is perfect number or not
2  def is_perfect_number(num):
3      if num < 1:
4          return False
5      divisors_sum = sum(i for i in range(1, num) if num % i == 0)
6      return divisors_sum == num
7
8  # Test cases for the is_perfect_number function
9  assert is_perfect_number(6) == True
10 assert is_perfect_number(10) == False
11 assert is_perfect_number(28) == True
12 assert is_perfect_number(1) == False
13 print("All test cases for is_perfect_number passed!")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\pniha\OneDrive\Desktop\AI AC> & C:\Users\pniha\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users
All test cases for is_perfect_number passed!
PS C:\Users\pniha\OneDrive\Desktop\AI AC>

```

Task 6 (Abundant Number Checker – Test Case Design)

- Function: Check if a number is abundant (sum of divisors > number).
- Test Cases to Design:
 - o Normal case: 12 → True, 15 → False.
 - o Edge case: 1.
 - o Negative number case.
 - o Large case: 945.

Requirement: Validate correctness with unittest

```
abundant num.py > ...
1  def Abundant_Number_Checker(num):
2      if num < 1:
3          return False
4      divisors_sum = sum(i for i in range(1, num) if num % i == 0)
5      return divisors_sum > num
6
7  import unittest
8  class TestAbundantNumberChecker(unittest.TestCase):
9      def test_abundant(self):
10         self.assertTrue(Abundant_Number_Checker(12))
11         self.assertTrue(Abundant_Number_Checker(15))
12         self.assertTrue(Abundant_Number_Checker(1))
13         self.assertFalse(Abundant_Number_Checker(-1))
14         self.assertTrue(Abundant_Number_Checker(945))
15
16  if __name__ == '__main__':
17      unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
1\Python\pythoncore-3.14-64\python.exe "c:/Users/pniha/OneDrive/Desktop/AI AC/abundant num.py"
F
=====
FAIL: test_abundant (__main__.TestAbundantNumberChecker.test_abundant)
=====
Traceback (most recent call last):
  File "c:/Users/pniha/OneDrive/Desktop/AI AC/abundant num.py", line 11, in test_abundant
    self.assertTrue(Abundant_Number_Checker(15))
    ~~~~~^~~~~~
AssertionError: False is not true

-----
Ran 1 test in 0.003s

FAILED (failures=1)
PS C:\Users\pniha\OneDrive\Desktop\AI AC> |
```

Task 7 (Deficient Number Checker – Test Case Design)

- Function: Check if a number is deficient (sum of divisors < number).
- Test Cases to Design:
 - o Normal case: 8 → True, 12 → False.
 - o Edge case: 1.
 - o Negative number case.
 - o Large case: 546.

Requirement: Validate correctness with pytest

```
Untitled-1.py X
Untitled-1.py > test_deficient_number_checker
1 def deficient_number_checker(num):
2     if num < 1:
3         return False
4     divisors_sum = sum(i for i in range(1, num) if num % i == 0)
5     return divisors_sum < num
6
7 def test_deficient_number_checker():
8     assert deficient_number_checker(8) == True
9     assert deficient_number_checker(12) == False
10    assert deficient_number_checker(1) == True
11    assert deficient_number_checker(546) == False
12    print("All test cases for deficient_number_checker passed!")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

=====
platform win32 -- Python 3.14.0, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\pniha\OneDrive\Desktop\AI AC\deficient
collected 1 item

Untitled-1.py . [100%]

===== 1 passed in 0.04s =====
PS C:\Users\pniha\OneDrive\Desktop\AI AC\deficient>
```

Task 8 :

Write a function LeapYearChecker and validate its implementation using 10 pytest test cases

```
01.py X
01.py > test_leap_year_checker
1 def LeapYearChecker(year):
2     if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
3         return True
4     return False
5
6 def test_leap_year_checker():
7     assert LeapYearChecker(2020) == True
8     assert LeapYearChecker(1900) == False
9     assert LeapYearChecker(2000) == True
10    assert LeapYearChecker(2021) == False
11    assert LeapYearChecker(2400) == True
12    assert LeapYearChecker(2100) == False
13    assert LeapYearChecker(1996) == True
14    assert LeapYearChecker(1999) == False
15    assert LeapYearChecker(1600) == True
16    print("All test cases for LeapYearChecker passed!")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

PS C:\Users\pniha\OneDrive\Desktop\AI AC\leap year> & C:\Users\pniha\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/pniha/OneDrive/Desktop
PS C:\Users\pniha\OneDrive\Desktop\AI AC\leap year> python -m pytest 01.py
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\pniha\OneDrive\Desktop\AI AC\leap year
collected 1 item

01.py . [100%]

===== 1 passed in 0.08s =====
PS C:\Users\pniha\OneDrive\Desktop\AI AC\leap year>
```

Task 9 :

Write a function SumOfDigits and validate its implementation using 7 pytest test cases.

```
02.py x
02.py > test_sum_of_digits
1 def SumOfDigits(num):
2     return sum(int(digit) for digit in str(abs(num)))
3
4 def test_sum_of_digits():
5     assert SumOfDigits(123) == 6
6     assert SumOfDigits(-456) == 15
7     assert SumOfDigits(0) == 0
8     assert SumOfDigits(9999) == 36
9     assert SumOfDigits(-1001) == 2
10    print("All test cases for SumOfDigits passed!")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\pniha\OneDrive\Desktop\AI AC\sumofdigits> & C:\Users\pniha\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/pniha/OneDrive/Desktop/AI AC/sumofdigits/02.py"
PS C:\Users\pniha\OneDrive\Desktop\AI AC\sumofdigits> python -m pytest 02.py
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\pniha\OneDrive\Desktop\AI AC\sumofdigits
collected 1 item

02.py . [100%]

===== 1 passed in 0.04s =====
PS C:\Users\pniha\OneDrive\Desktop\AI AC\sumofdigits>
```

Task 10 :

Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.

```
03.py x
03.py > test_sort_numbers
1 def SortNumbers(numbers):
2     n = len(numbers)
3     for i in range(n):
4         for j in range(0, n-i-1):
5             if numbers[j] > numbers[j+1]:
6                 numbers[j], numbers[j+1] = numbers[j+1], numbers[j]
7     return numbers
8
9 def test_sort_numbers():
10    assert SortNumbers([5, 2, 9, 1, 5, 6]) == [1, 2, 5, 5, 6, 9]
11    assert SortNumbers([]) == []
12    assert SortNumbers([3]) == [3]
13    assert SortNumbers([3, 2]) == [2, 3]
14    assert SortNumbers([1, 2, 3]) == [1, 2, 3]
15    assert SortNumbers([3, 2, 1]) == [1, 2, 3]
16    assert SortNumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
17    assert SortNumbers([1, 1, 1, 1]) == [1, 1, 1, 1]
18    assert SortNumbers([9, 8, 7, 6, 5]) == [5, 6, 7, 8, 9]
19    assert SortNumbers([10, 9, 8, 7, 6]) == [6, 7, 8, 9, 10]
20    assert SortNumbers([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]
21    assert SortNumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
22    assert SortNumbers([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]
23    assert SortNumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
24    assert SortNumbers([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]
25    print("All test cases for SortNumbers passed!")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
rootdir: C:\Users\pniha\OneDrive\Desktop\AI AC\sort numbers
collected 1 item

03.py . [100%]

===== 1 passed in 0.05s =====
PS C:\Users\pniha\OneDrive\Desktop\AI AC\sort numbers>
```

Task 11 :

Write a function ReverseString and validate its implementation using 5 unittest test cases


```
04.py  anagram checker.py  armstrong.py
C:\Users\pniha> OneDrive\OneDrive\Desktop\AI AC> armstrong.py > ...
1  def ArmstrongChecker(num):
2      num_str = str(num)
3      num_digits = len(num_str)
4      armstrong_sum = sum(int(digit) ** num_digits for digit in num_str)
5      return armstrong_sum == num
6
7  import unittest
8  class TestArmstrongChecker(unittest.TestCase):
9      def test_armstrong_checker(self):
10         self.assertTrue(ArmstrongChecker(153))
11         self.assertTrue(ArmstrongChecker(9474))
12         self.assertFalse(ArmstrongChecker(123))
13         self.assertTrue(ArmstrongChecker(0))
14         self.assertTrue(ArmstrongChecker(1))
15         self.assertFalse(ArmstrongChecker(10))
16         self.assertTrue(ArmstrongChecker(375))
17         self.assertTrue(ArmstrongChecker(371))
18
19 if __name__ == '__main__':
20     unittest.main()

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
=====
FAIL: test_armstrong_checker (__main__.TestArmstrongChecker.test_armstrong_checker)
-----
Traceback (most recent call last):
  File "c:\Users\pniha\OneDrive\Desktop\AI AC\armstrong.py", line 17, in test_armstrong_checker
    self.assertTrue(ArmstrongChecker(375))
    ~~~~~^~~~~~
AssertionError: False is not true
-----
Ran 1 test in 0.001s

FAILED (failures=1)
PS C:\Users\pniha\AppData\Local\Programs\Microsoft VS Code> 
```